

Prática 3 - Servidor TCP concorrente com threads

1. Qual problema a utilização de threads resolve neste servidor?

O uso de threads resolve o problema do servidor TCP que aceitava apenas uma conexão por vez. A estratégia permite que o servidor atenda múltiplos clientes simultaneamente, introduzindo concorrência.

2. Quais riscos existem ao criar uma thread para cada cliente sem limite?

Criar uma thread para cada conexão, sem um teto de limite estrito, pode esgotar rapidamente os recursos do sistema operacional (memória RAM e ciclos de CPU), tornando a aplicação ou a máquina inteira vulnerável a travamentos e ataques de negação de serviço (DoS/DDoS).

3. O que significa `thread.daemon = True` neste contexto?

Significa que a thread funcionará em segundo plano (background). Dessa forma, quando a thread principal do programa (o servidor em si) for encerrada, o sistema operacional irá finalizar automaticamente todas as threads *daemon* em vez de esperar que elas concluam suas rotinas, evitando que processos fiquem "presos" escutando portas ativamente na máquina.

4. Por que a saída no terminal do servidor pode aparecer intercalada?

Porque as múltiplas threads estão em execução concorrente e compartilham o mesmo recurso do sistema padrão (o stdout do console). Como não há um *lock* (bloqueio estruturado) na função `print()` para sincronização rigorosa, o escalonador do SO pode alternar o uso da CPU para uma thread diferente no exato meio da escrita de outra.

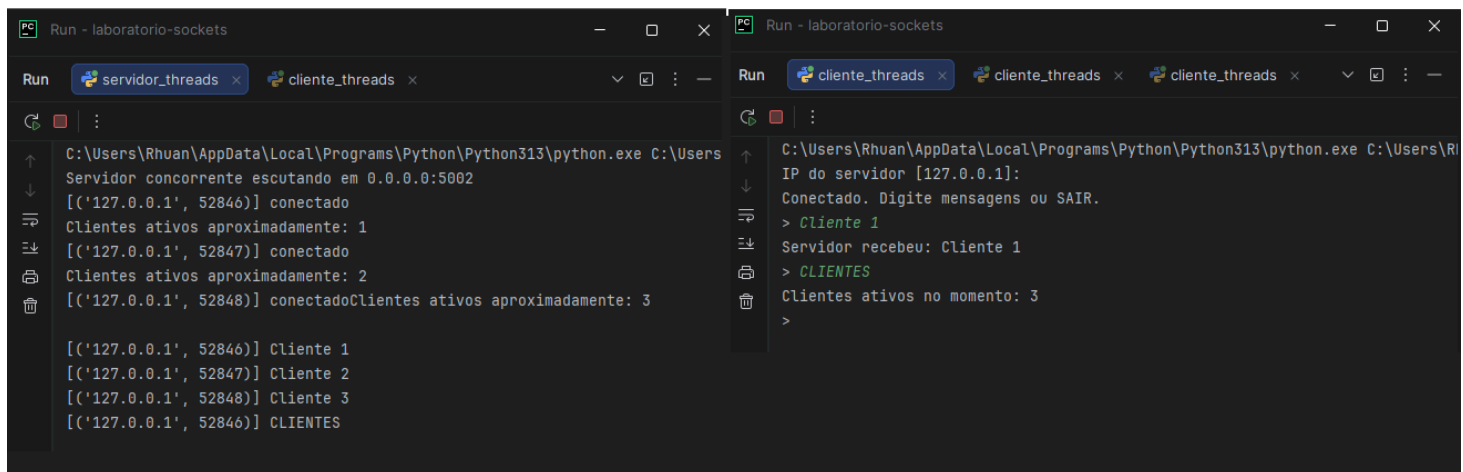
5. Como esse modelo se compara a um servidor baseado em eventos, como `select` ou `asyncio`?

O modelo de threads (um-para-um) cria forte sobrecarga (overhead) com a constante troca de contexto (context switch) por parte do SO, além de alocar blocos pesados de memória por thread. O modelo de eventos é projetado com *I/O não bloqueante* e usa, normalmente, apenas uma thread (event loop) que atende todas as conexões simultaneamente realizando a multiplexação. Eventos são superiores para alta escalabilidade e milhares de clientes simultâneos.

Variação Implementada

A variação escolhida para esta prática foi a criação do comando especial `CLIENTES`, que retorna a quantidade aproximada de clientes ativos no servidor. Para que isso funcionasse, foi adicionada uma estrutura condicional dentro da função `atender\_cliente` que intercepta o texto enviado pelo usuário. Quando esse comando específico é detectado, em vez de apenas confirmar o recebimento da mensagem, o servidor aciona o método `threading.active\_count() - 1` para calcular quantas threads secundárias estão em execução (descontando a thread principal que continua escutando a porta) e devolve esse número ao cliente, criando um mecanismo simples e eficaz para monitorar o nível de concorrência da aplicação em tempo real.

Print 3 clientes simultaneos



```
Run - laboratorio-sockets
Run  servidor_threads x cliente_threads x
C:\Users\Rhuan\AppData\Local\Programs\Python\Python313\python.exe C:\Users
Servidor concorrente escutando em 0.0.0.0:5002
[('127.0.0.1', 52846)] conectado
Clientes ativos aproximadamente: 1
[('127.0.0.1', 52847)] conectado
Clientes ativos aproximadamente: 2
[('127.0.0.1', 52848)] conectado Clientes ativos aproximadamente: 3

[('127.0.0.1', 52846)] Cliente 1
[('127.0.0.1', 52847)] Cliente 2
[('127.0.0.1', 52848)] Cliente 3
[('127.0.0.1', 52846)] CLIENTES

Run - laboratorio-sockets
Run  cliente_threads x cliente_threads x cliente_threads x
C:\Users\Rhuan\AppData\Local\Programs\Python\Python313\python.exe C:\Users\RI
IP do servidor [127.0.0.1]:
Conectado. Digite mensagens ou SAIR.
> Cliente 1
Servidor recebeu: Cliente 1
> CLIENTES
Clientes ativos no momento: 3
>
```